The Journal of Open Source Software

JSBSim, An Open Source Flight Dynamics Software Library

Jon S. Berndt [1], **Bertrand Coconnier** [2], **Agostino De Marco** [3], **and Sean McLeod** [4]

1 Independent Developer, United States **2** Independent Developer, France **3** Professor of Flight Mechanics, University of Naples Federico II, Italy **4** Independent Developer, South Africa

DOI: 10.21105/joss.11129

Software

- Review
- Repository
- Archive

Editor: Kyle Niemeyer

Reviewers:

- @milanofthe
- @t3r

Submitted: 23 May 2026
Published: 22 October 2026

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

JSBSim is an open-source, platform-independent, data-driven flight dynamics software library for aerospace research, simulation development, and education. It provides high-fidelity modeling of aerospace vehicle dynamics, and includes propulsion, control systems, aerodynamics, and other subsystem models, as well as environmental models such as atmosphere, gravity, and geodesy.

JSBSim can be invoked from the command line — running faster than real time for scripted flight scenarios, Monte Carlo studies, or AI training — or it can be integrated into an interactive simulator codebase such as FlightGear, Unreal Engine, or Outerra and run in real time.

Features include:

- Rigid body dynamics with support for 6-degrees-of-freedom (6-DoF) simulations.
- Quaternion-based computation of the aircraft attitude to avoid the gimbal lock of Euler angles.
- Accurate environment models (geodesy, atmosphere, rotational planet effects).
- Fully configurable input model characteristics and output logging.
- Developed in standard-compliant C++17, with bindings for Python, and MATLAB (includes a Simulink S-Function).

JSBSim has been in active development for almost 30 years.

Statement of Need

JSBSim emerged from the need for an open, modern, clean, extensible flight dynamics model (FDM). Aerospace researchers, instructors, and engineers often need an FDM that is scientifically credible and openly accessible. Existing FDMs are either proprietary, tightly integrated with specific simulators, or lack extensibility for custom modeling, automated testing, or integration into research pipelines. Others have been in development for so long that they have become difficult to adapt or even to understand. JSBSim fills this gap by offering a standalone, open-source FDM with a clear architecture, straightforward and predictable behavior, and a long history in academic, government, and open-source projects.

The JSBSim XML-based model definitions support validation, and scriptable running supports testing and reproducibility.

State of the Field

In the research community, several tools are currently used to model flight dynamics, but they are often unavailable for public use or present significant limitations for automated research or high-fidelity academic studies. The Trick simulation environment, developed at NASA Johnson Space Center, was released as open source software in 2015. POST2 — successor to POST, Program to Optimize Simulated Trajectories, developed over 50 years ago — is still used internally within NASA to this day, as a trusted and well-verified tool. Within the open-source and free-to-use landscape, YASim (also used in FlightGear) offers a quick-solving approach where the FDM is automatically generated from geometry and performance points using Blade Element Theory. While efficient for rapid prototyping, it generates plausible but low-fidelity models. X-Plane also uses a fast, low-fidelity aerodynamic model, but it cannot run natively in a headless mode. Its closed-source nature makes it difficult to extend or integrate deeply into custom research pipelines without complex, third-party interfaces. Engineering-focused tools like the MATLAB Aerospace Toolbox provide high-fidelity components but are proprietary and require the MATLAB/Simulink ecosystem.

Unique Scholarly Contribution

JSBSim's unique contribution lies in providing a standalone traditional coefficient-based, "architecture-light," FDM that bridges the gap between the alternatives mentioned above. Unlike YASim and X-Plane, JSBSim primarily uses stability derivatives to model vehicle aerodynamics. Stability derivatives can be constant or functionally dependent on other state and input variables. However, the use of stability derivatives is not the only option available to FDM authors. The XML-based metalanguage provided by JSBSim allows for the use of general, nonlinear, and multidimensional lookup tables to model any type of behavior within the FDM.

By providing a complete, turnkey solution, JSBSim allows researchers to bypass the complexity of building a simulation engine from scratch and focus entirely on their specific area of study. Furthermore, for research topics that require modeling beyond the standard stability derivative approach, JSBSim's `external_reactions` feature offers the flexibility to interface with any external aerodynamic or propulsive simulation (or co-simulation) method, ensuring the library remains extensible for even the most unconventional aerospace designs.

Software Design

JSBSim was designed from the ground up with several features in mind (Berndt, 2004). One was to make the codebase easily comprehensible and expandable, and another was to completely separate the characteristics of a specific vehicle from a completely generic codebase. This was done in part to keep possibly proprietary information out of the codebase. With all specific model characteristics contained in data files, there is no need to recompile the code to model a different vehicle, or changes to the vehicle characteristics. This is a key design feature of JSBSim, which allows users to define an entire FDM using XML files — unlike, for example, LaRCSim, where modifying aircraft parameters requires writing and re-compiling C code.

JSBSim scales to highly complex aerospace vehicles — even rockets with GNC systems and large aerodynamic databases derived from wind tunnel testing — all specified through data files alone. In fact, the input metalanguage offers great flexibility in terms of defining properties within a specific FDM, with the availability of a large number of mathematical operators, interpolating functions from data arranged in tabular form, and access to the aircraft's metrics and state via the property system. In the example of Figure 1, the pitch moment due to elevator (linearly dependent on its deflection δ_e) is defined in terms of a control power coefficient $C_{m\delta_e}$, which is a function of Mach number M_∞.

```
<function name="aero/PitchMoment_elevator">
    <description>Pitch moment due to elevator</description>
    <product>
        <property> aero/qbar-psf    </property>              ⟵ q̄∞ ·
        <property> metrics/Sw-sqft  </property>              ⟵ S ·
        <property> metrics/cbarw-ft </property>              ⟵ c̄w ·
        <property> fcs/elevator-pos-rad</property>           ⟵ δe ·
        <table> <!-- 1D tabular function -->                 ⟵ Cmδe = fcn(M∞)
            <independentVar> velocities/mach </independentVar>
            <tableData> <!-- lookup table: Cmde coefficient -->
                0.0 -1.20
                1.0 -1.00
                2.0 -0.30
            </tableData>
        </table>
    </product>
</function>
```

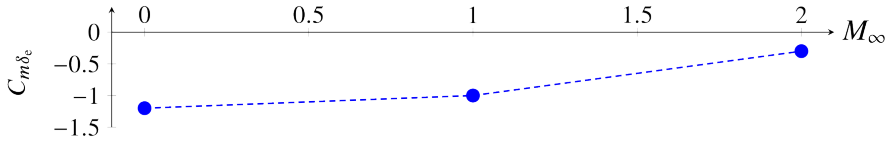


Figure 1: Pitching control power coefficient as a tabulated function of Mach number. See `<table>` element in the above snippet.

The more common execution from the command line involves running from a script, which interacts with the simulation by modifying properties based on conditional logic. While JSBSim handles the continuous physics of flight, the script acts as a state-machine-driven mission controller, providing the discrete logical transitions required to navigate complex flight scenarios. JSBSim scripts are coded as XML-based input files, with their specific metalanguage.

A Selected Example of Use in Research: Flight Load Assessment for Light Aircraft Flying near Wind Turbine Wake

JSBSim has been used by Varriale et al. (2018) to assess the flight loads on light aircraft flying through or nearby wind turbine wakes. For this research JSBSim's autopilot system has been used to simulate realistic pilot behavior during navigation, see Figure 2.

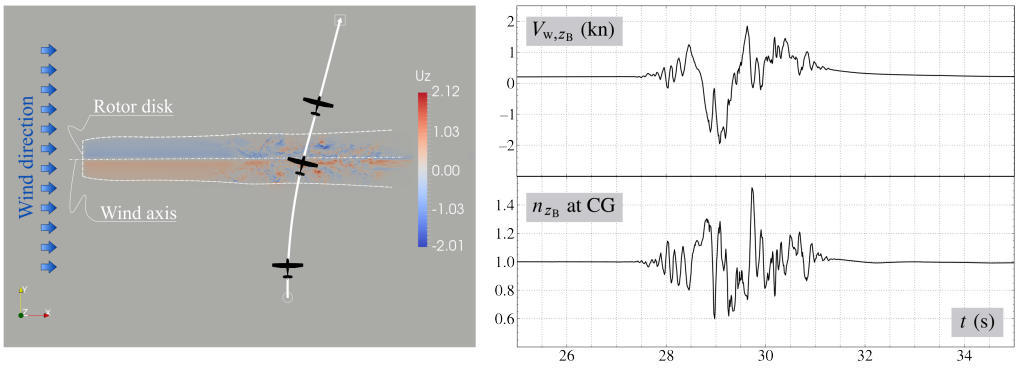


Figure 2: Simulation results adapted from Varriale et al. (2018). Example from a population of simulated scenarios in the presence of CFD calculated wind and turbulence fields. JSBSim's autopilot system has been used to simulate realistic pilot behavior during navigation. (Left) Top view of an airplane flying over a wind-turbine wake. (Right) Time histories of the normal relative-wind component (along z-body axis) induced by the turbine wake, and the normal load factor. Based on these results, preliminary guidelines and recommendations on safe encounter distances have been provided for general aviation aircraft when flying in the proximity of wind farms.

Implementation and Engineering Practices

A key requirement of an FDM is accuracy. The JSBSim equations of motion were verified through comparison with other similar flight simulation applications. The NASA Engineering Safety Center undertook an effort in 2015 to develop a set of check cases that could serve as a basis for comparing time-history data across simulations. JSBSim was included in this effort as the only non-NASA simulation software (Murri et al., 2015).

The library also leverages the broader open-source ecosystem by integrating mature and specialized components. For XML parsing, JSBSim relies on Expat, a foundational and industry-standard library in the open-source community. Additionally, complex geodesic calculations required for high-fidelity trajectory modeling on the WGS84 oblate spheroid are performed using Charles Karney's GeographicLib, ensuring precision in geospatial positioning and navigation (Karney, 2025).

JSBSim adheres to modern open-source Quality Assurance standards through an extensive Continuous Integration and Continuous Deployment.

Research Impact Statement

JSBSim is used across a broad range of aerospace applications, including flight control development, UAV research, aircraft design studies, and simulation-based testing. Its use in academic and industry research has resulted in over 1000 citations as per Google Scholar, and it has been integrated into several popular flight simulators and research platforms. In the existing scientific literature, the key works on JSBSim are those by Berndt (2004), De Marco et al. (2007), Berndt & De Marco (2009), and Murri et al. (2015).

Examples of use cases include:

- Modeling flight dynamics within a full-featured flight simulator, such as FlightGear, MIXR (Mixed Reality Simulation Platform) (formerly known as OpenEaagles), the Outerra world simulator, or Epic Games' Unreal Engine 5.
- Control system design. See the articles by Vogeltanz (2018) and Vogeltanz (2020).

- Reinforcement learning research, where JSBSim is used as the environment in which an agent learns to control an aircraft. One example being its use in the DARPA Virtual Air Combat Competition. See also the works by De Marco et al. (2023), Pope et al. (2023), Chen et al. (2026), and Richter & Calix (2021).
- SITL (Software In The Loop) Drone autopilot testing: ArduPilot, PX4 Autopilot, Paparazzi.
- UAV modeling. See Kamal et al. (2016), Cereceda et al. (2019).
- Rocket trajectory simulations. See Gomez & Miikkulainen (2003), Braun et al. (2006), Kenney & Croom (2011).
- Sensor assessment and Human Factor. See Zhang & McGovern (2010), McAnanama & Marsden (2018).
- Simulation integration. See Park et al. (2008), Nicolosi et al. (2018), Saber et al. (2025).
- CPU performance benchmarking. JSBSim has been included in the SPEC CPU 2026 (Madhav et al., 2026), a widely recognized benchmark suite designed to measure the performance of a computer's processor, memory, and compiler efficiency using compute-intensive workloads.

JSBSim's versatility is further expanded by its blossoming Python ecosystem, which has seen over one million cumulative downloads across PyPI and Conda. While its established C++ integration remains fundamental, there is a significant growth in users leveraging Python's ubiquity in the research, engineering, and AI communities. This trend is reflected in a growing number of applications that utilize interactive notebooks to provide a didactic interface, effectively lowering the barrier to entry. Moreover, Python enables a more dynamic approach to simulation, allowing users to complement traditional XML scripting with complex programmatic scenarios and to develop sophisticated aerodynamic or propulsion models. A great potential of the JSBSim Python API is confirmed, for instance, by its straightforward usability within applications based on PathSim (Rother, 2025) and its companion PathView; the latter being a Python native environment with a visual toolbox capable of modeling simulation workflows, as noncommercial alternative to MATLAB/Simulink or Modelica.

Acknowledgements

JSBSim is currently being maintained and developed by the authors. Initial architecture and development was done by Jon S. Berndt (JSB), with major contributions from Tony Peden, David Megginson, David Culp, and Curt Olson.

AI Usage Disclosure

This manuscript was prepared also with support of AI-based tools, basically to enhance clarity and refine language. The authors remain fully responsible for the content of the manuscript. In terms of coding, data generation and scientific claims — due to the fact that JSBSim largely predates the advent of AI — most of the software development has been made by humans and all the contributions are reviewed by humans.

References

Berndt, J. S. (2004). JSBSim: An open source flight dynamics model in C++. *AIAA Modeling and Simulation Technologies Conference and Exhibit*. https://doi.org/10.2514/6.2004-4923

Berndt, J. S., & De Marco, A. (2009). Progress on and usage of the open source flight dynamics model software library, JSBSim. *AIAA Modeling and Simulation Technologies Conference*, 1–17. https://doi.org/10.2514/6.2009-5699

Braun, R. D., Wright, H. S., Croom, M. A., Levine, J. S., & Spencer, D. A. (2006). Design of the ARES Mars airplane and mission architecture. *Journal of Spacecraft and Rockets*, *43*(5), 1026–1034. https://doi.org/10.2514/1.17956

Cereceda, O., Rolland, L., & O'Young, S. (2019). Giant Big Stik R/C UAV computer model development in JSBSim for sense and avoid applications. *Drones*, *3*(2). https://doi.org/10.3390/drones3020048

Chen, Y. H., Chiang, M. H., & Peng, C. C. (2026). Physics-informed reinforcement learning for air-to-air F16 target aiming. *Advanced Engineering Informatics*, *69*. https://doi.org/10.1016/j.aei.2025.103988

De Marco, A., D'Onza, P. M., & Manfredi, S. (2023). A deep reinforcement learning control approach for high-performance aircraft. *Nonlinear Dynamics*, *111*, 17037–17077. https://doi.org/10.1007/s11071-023-08725-y

De Marco, A., Duke, E. L., & Berndt, J. S. (2007). A general solution to the aircraft trim problem. *AIAA Modeling and Simulation Technologies Conference and Exhibit*. AIAA modeling and simulation technologies conference and exhibit, Hilton Head, South Carolina. https://doi.org/10.2514/6.2007-6703

Gomez, F. J., & Miikkulainen, R. (2003). Active guidance for a finless rocket using neuroevolution. *Genetic and Evolutionary Computation — GECCO 2003*, *2724*, 2084–2095. https://doi.org/10.1007/3-540-45110-2_105

Kamal, A. M., Bayoumy, A. M., & Elshabka, A. M. (2016). Modeling and flight simulation of unmanned aerial vehicle enhanced with fine tuning. *Aerospace Science and Technology*, *51*, 106–117. https://doi.org/10.1016/j.ast.2016.01.022

Karney, C. (2025). *GeographicLib 2.7*. Zenodo. https://doi.org/10.5281/zenodo.18011538

Kenney, P. S., & Croom, M. A. (2011). Flight simulation of ARES in the Mars environment. *NASA Tech Briefs*, *35*(8). https://ntrs.nasa.gov/citations/20120006630

Madhav, M., Lee, A., Mejia, A., Moore, B., Soppadandi, C., Cambly, C., Müllner, C., Bowers, D., Reiner, D., Bakhvalov, D., Zhao, D., Voth, D., Xue, F., Silber-Chaussumier, F., Bucek, J., Southern, J., Liu, J., Himer, J., Henning, J., … Todorovski, Z. (2026). *SPEC CPU: The next generation*. https://arxiv.org/abs/2605.01575

McAnanama, J. G., & Marsden, G. (2018). An open source flight dynamics model and IMU signal simulator. *2018 IEEE/ION Position, Location and Navigation Symposium (PLANS)*, 874–881. https://doi.org/10.1109/PLANS.2018.8373465

Murri, D. G., Jackson, E. B., & Shelton, R. O. (2015). *Check-cases for verification of 6-degree-of-freedom flight vehicle simulations* (NASA Technical Memorandum NASA/TM-2015-218675/Vol. I). NASA Langley Research Center. https://doi.org/10.64631/IGUD7952

Nicolosi, F., De Marco, A., Sabetta, V., & Della Vecchia, P. (2018). Roll performance assessment of a light aircraft: Flight simulations and flight tests. *Aerospace Science and Technology*, *76*, 471–483. https://doi.org/10.1016/j.ast.2018.01.041

Park, H., Kim, D.-H., Kim, J.-G., & Chang, C.-H. (2008). Experimental evaluation of unmanned aerial vehicle system software based on the TMO model. *Journal of Computing Science and Engineering*, *2*(4), 357–374. https://doi.org/10.5626/jcse.2008.2.4.357

Pope, A. P., Ide, J. S., Mićović, D., Díaz, H., Twedt, J. C., Alcedo, K., Walker, T. T., Rosenbluth, D., Ritholtz, L., & Javorsek, D. (2023). Hierarchical reinforcement learning for air combat at DARPA's AlphaDogfight trials. *IEEE Transactions on Artificial Intelligence*,

4(6), 1371–1385. https://doi.org/10.1109/TAI.2022.3222143

Richter, D. J., & Calix, R. A. (2021). Qplane: An open-source reinforcement learning toolkit for autonomous fixed wing aircraft simulation. *Proceedings of the 12th ACM Multimedia Systems Conference*, 261–266. https://doi.org/10.1145/3458305.3478446

Rother, M. (2025). PathSim - a system simulation framework. *Journal of Open Source Software*, *10*(109), 8158. https://doi.org/10.21105/joss.08158

Saber, M. A., Zakaria, M. Y., & Kamal, A. M. (2025). Integration of JSBSim and Unreal Engine for flight simulator development: A case study on the Cessna T-37. *Journal of Physics: Conference Series*, *3070*(1), 1–14. https://doi.org/10.1088/1742-6596/3070/1/012009

Varriale, C., De Marco, A., Daniele, E., Schmidt, J., & Stoevesandt, B. (2018). Flight load assessment for light aircraft landing trajectories in windy atmosphere and near wind farms. *Aerospace*, *5*(2). https://doi.org/10.3390/aerospace5020042

Vogeltanz, T. (2020). Control system designer for JSBSim with algorithm of automatic Ziegler-Nichols sustained-oscillation method. In T. Simos & C. Tsitouras (Eds.), *AIP conference proceedings* (Vol. 2293). AIP Publishing. https://doi.org/10.1063/5.0026623

Vogeltanz, T. (2018). Development of control system designer for JSBSim FDM. In T. Simos & C. Tsitouras (Eds.), *AIP conference proceedings* (Vol. 1978). AIP Publishing. https://doi.org/10.1063/1.5043721

Zhang, Y., & McGovern, S. M. (2010). Mathematical models for human pilot maneuvers in aircraft flight simulation. *Proceedings of the ASME 2009 International Mechanical Engineering Congress and Exposition*, *13*, 35–39. https://doi.org/10.1115/IMECE2009-10503